Universidad Autónoma de Madrid

Departamento de Ing. Informática

Sistemas Informáticos II
Práctica - 0

Daniel Hernández Lobato

Registro de Cambios

Versión[1]	**Fecha**	**Autor**	**Descripción**
1.0	10.12.2024	DHL	Primera versión.

[1]La asignación de versiones se realizan mediante 2 números $X.Y$. Cambios en Y indican aclaraciones, descripciones más detalladas de algún punto o traducciones. Cambios en X indican modificaciones más profundas que o bien varían el material suministrado o el contenido de la práctica.

Índice

1. Objetivos **4**

2. Entorno **4**
2.1. Búsqueda de Comandos Introducidos en la Shell 5

3. El Software de Virtualización VirtualBox **5**
3.1. Máquinas Vrituales 7
3.2. Arrancado de la Máquina Virtual 7
3.3. Acceso Remoto a la Máquina Virtual 7
3.4. Conexión a la Máquina Virtual 8
3.5. Apagado de la Máquina Virtual 8
3.6. Transportar la Máquina Virtual 8

4. Django **9**
4.1. Componentes de una Aplicación Web de Django 9
4.2. Mapeador URL (urls.py) 11
4.3. Manejar la Petición (views.py) 12
4.4. Definir Modelos de Datos (models.py) 12
4.5. Consultar Datos (views.py) 13
4.6. Renderización de los Datos (plantillas HTML) 14

5. Gunicorn **15**

6. PostgreSQL **16**
6.1. DBeaver 17

7. Apache Jmeter **20**

8. Servidor Apache **20**
8.1. Apache Como Balanceador de Carga 22

9. Instalación del Entorno en un PC Propio **23**
9.1. Virtual Box 23
9.2. Python 3 24
9.3. Django, Gunicorn y Demás Paquetes Python 25
9.4. Jakarta Jmeter 26
9.5. DBeaver 26
9.6. Otras Herramientas 27

10. Entrega **27**

1. Objetivos

En esta primera práctica de Sistemas Informáticos se proporcionará una pequeña introducción al entorno de software que se va a utilizar a lo largo del curso. Se verá una pequeña introducción o repaso a los siguientes aspectos:

- **Django** como un framework de desarrollo web de código abierto, escrito en Python, que respeta el patrón de diseño conocido como modelo–vista–controlador (MVC).
- **Gunicorn** como servidor Python WSGI HTTP para sistemas unix.
- El sistema de gestión de bases de datos **PostgreSQL**. De este las siguientes interfaces:
  - En línea de comandos: psql, createdb, dropdb,... para conexión, creación y borrado de bases de datos
  - Como interfaz gráfica: **DBeaver**.
- **Jakarta JMeter** como herramienta gráfica de generación de carga y test.
- El servidor web **Apache** como balanceador de carga externo a django.
- El entorno de virtualización **Virtualbox**, incluyendo una máquina virtual Ubuntu Server con todas las aplicaciones anteriores instaladas.

No es necesario entregar nada al completar la práctica. El objetivo es sólo que el estudiante sea capaz de reconocer, instalar y utilizar cada uno de estos elementos. La dedicación estimada para esta práctica es de 2 horas presenciales y 3 horas no presenciales.

2. Entorno

Para realizar las prácticas en el laboratorio se podrá utilizar únicamente la partición de **Linux**. Los programas que es necesario utilizar ya se encuentran instalados en dicha partición.

A lo largo de las prácticas se distinguirá entre los siguientes entornos, habituales en aplicaciones empresariales:

- **Entorno de desarrollo:** El entorno software donde trabajan los desarrolladores de las aplicaciones y realizan las pruebas funcionales de las mismas. En nuestro caso, el entorno de desarrollo son los PCs de las aulas (x86 con Ubuntu Desktop 22.04 LTS), nuestro propio PC (si dispone de Ubuntu Desktop 22.04 LTS) o una máquina virtual de nuestro PC (con Ubuntu Desktop 22.04 LTS).
- **Entorno de producción:** Es el entorno donde se ejecuta la aplicación para prestar el servicio para el que está diseñada a los usuarios finales. Normalmente es un entorno remoto, con fuertes restricciones de seguridad en el acceso y cambios, ubicado en la infraestructura de producción del cliente, y con grandes capacidades de monitorización y gestión. En estas prácticas no contamos con este entorno, pero lo simularemos empleando una máquina virtual Virtual Box ejecutando Ubuntu 24 LTS dentro del mismo PC.

2.1. Búsqueda de Comandos Introducidos en la Shell

A lo largo de las prácticas se hará un uso extensivo de la consola de comandos o shell de Linux. Para simplificar la introducción de comandos resulta muy útil la búsqueda de comandos ya introducidos en el historial de comandos. Para ello, únicamente hay que pulsar las teclas ctrl+r. Al empezar a escribir el comando, nos aparecerá el último comando completo ya introducido que empieza por los mismos caracteres ya introducidos. Al pulsar ctrl+r de nuevo, aparecerá el siguiente comando introducido que es compatible con el patrón de búsqueda, etc.

```
(reverse-i-search)‘ls’: ls /home/si2
```

3. El Software de Virtualización VirtualBox

Este software de virtualización es usado profesionalmente para virtualizar sistemas operativos completos, es decir, ejecutar concurrentemente varias instancias

separadas del sistema operativo sobre el mismo hardware con mínimo impacto sobre el rendimiento del mismo. Distinguimos dos entidades fundamentales:

- **Sistema anfitrión o host**: aquel bajo el que se está ejecutando el propio software de virtualización (en el caso de los laboratorios: Ubuntu Desktop 22.04 LTS)
- **Sistema invitado o guest**: aquel que es virtualizado y ejecutado dentro de la máquina virtual. En nuestro caso se trata de una instalación de Ubuntu 24 con todo lo necesario para poder realizar las prácticas de esta asignatura. Habitualmente nos referiremos a éste simplemente como la Máquina Virtual o VM.

La manera habitual de conectar con la VM es utilizando TCP/IP. Ésta incluye interfaces de red *virtuales* para realizar esta función. Se distinguen tres tipos:

- **Interfaz host-only**: una interfaz de muy bajo overhead que permite la conexión únicamente con el PC host. En nuestras prácticas no la usaremos por simplicidad y porque no es realista en cuanto a su rendimiento
- **Interfaz NAT**: permite que la VM se conecte al exterior utilizando la propia interfaz del PC host mediante traducción de direcciones de red. Sin embargo, en principio, no permite la operación inversa (conexiones entrantes, desde el exterior hacia la VM). Para poder conectarse a la VM es necesario realizar un mapeo de puertos del PC host a puertos de la VM. Nuestra configuración incluye una de estas 'tarjetas' (enp0s3) para su propio mantenimiento de actualizaciones y acceso remoto mediante el mapeo de puertos, que será descrito más adelante.
- **Interfaz puente o bridge**: es una interfaz de red con un funcionamiento idéntico a la del propio PC. Utiliza su propia dirección MAC (configurable por el usuario) y todos los PCs (virtuales o no) que compartan segmento de red podrán ver dicha dirección MAC en sus tablas arp y, por tanto, acceder a la máquina virtual. No usaremos esta interfaz de red ya que en los laboratorios los cortafuegos filtran el tráfico cuyas direcciones MAC no han sido registradas previamnete.

3.1. Máquinas Vrituales

En esta sección se describe el proceso de uso de las VM que emplearemos en las prácticas de la asignatura. Para ello, se proporcionan VM pre-creadas. En principio, en las prácticas trabajaremos con 1, 2 y 3 VM. Estas se pueden descargar desde moodle a través del enlace allí facilitado. **Todas las máquinas virtuales tienen creado un usuario llamado "si2", cuya contraseña es también "si2".** Se recomienda cambiar dicha contraseña con el comando "passwd".

3.2. Arrancado de la Máquina Virtual

Para arrancar la VM basta con ejecutar virtualbox y abrir la VM correspondiente. Debido a que la transferencia de datos entre una unidad USB y el PC es muy lento, se recomienda copiar la VM al PC host antes de arrancarla y no arrancarla desde una unidad USB.

3.3. Acceso Remoto a la Máquina Virtual

Para acceder a la VM utilizaremos la interfaz NAT y el mapeo de puertos que ha generado el script de configuración, al crear las VM. Dicho mapeo de puertos aparece descrito en la tabla 2, dependiendo del número de VM que hayamos arrancado. Este mapeo es válido para la VM número 1. Para las VM número 2 y 3 el mapeo es equivalente, pero empezando los puertos del PC host por 2 y 3, en lugar de 1, respectivamente.

Tabla 2: Tabla de mapeo de puertos de la VM número 1.

Servicio	**Puerto Host**	**Puerto VM**
ssh	12022	22
apache	18080	80
postgresql	15432	5432
gunicorn	18000	8000
rabbitmq	15672	5672

3.4. Conexión a la Máquina Virtual

Podemos interactuar con la VM directamente desde la ventana correspondiente de VirtualBox. Al pulsar en ella, capturará el foco del teclado y el ratón. Para recuperarlo, basta con pulsar **la tecla ctrl derecho**. Sin embargo, no es recomendable acceder a la VM de esta forma, y se recomienda su acceso mediante ssh, descrito en el siguiente párrafo.

Una vez hemos configurado la VM nos podemos conectar a ella por SSH mediante el comando:

```
ssh si2@ip_pc_host -p 12022
```

donde *ip_pc_host* es la dirección IP del PC host que contiene la VM. Esta ip se puede obtener mediante el comando ifconfig, que muestra las interfaces de red de dicho PC. En caso de estar en el mismo host, basta con usar el nombre *localhost*.

Si todo ha ido bien, veremos como, tras introducir la contraseña, nos aparece un prompt de la shell.

3.5. Apagado de la Máquina Virtual

Una vez se ha terminado de trabajar con la VM es recomendable apagarla. Se puede apagar con el comando

```
sudo shutdown now
```

Tras unos segundos se cerrará la ventana de la VM.

3.6. Transportar la Máquina Virtual

La VM puede ser transportada recomendadamente en un medio externo (USB) del propio alumno. Para ello basta con que se copie la carpeta de la VM al medio externo y se recupere de vuelta cuando sea necesario. Importante: Antes de copiar la carpeta con la máquina virtual, ésta debe estar parada. Se puede reducir el tamaño de la imagen comprimiéndola, por ejemplo con gzip.

4. Django

Django es un framework de desarrollo web de código abierto escrito en Python, que sigue el patrón de diseño modelo–vista-plantilla (template) (MTV). Fue creado inicialmente para gestionar sitios web de noticias de la World Company en Lawrence, Kansas, y se liberó al público bajo una licencia BSD en julio de 2005. El nombre *Django* rinde homenaje al guitarrista de jazz gitano Django Reinhardt.

En junio de 2008, se anunció que la recién formada Django Software Foundation asumiría la gestión del proyecto. El objetivo principal de Django es facilitar la creación de sitios web complejos, enfatizando la reutilización y extensibilidad de componentes, el desarrollo rápido y el principio DRY ('Don't Repeat Yourself' o 'No te repitas'). Python se utiliza en todos los componentes del framework, incluyendo configuraciones, archivos y modelos de datos.

Django ofrece una serie de características que facilitan el desarrollo rápido de páginas orientadas a contenidos. Por ejemplo, proporciona una aplicación administrativa incorporada que permite la creación, actualización y eliminación de objetos de contenido, llevando un registro de todas las acciones realizadas sobre cada uno, y ofrece una interfaz para gestionar usuarios y grupos de usuarios con asignación detallada de permisos.

Además, Django soporta múltiples bases de datos y servidores web, lo que lo hace versátil para diversos entornos de desarrollo. Su arquitectura modular permite integrar aplicaciones de terceros, ampliando sus funcionalidades según las necesidades del proyecto.

En resumen, Django es una herramienta poderosa para el desarrollo de aplicaciones web, reconocida por su eficiencia, flexibilidad y la sólida comunidad que la respalda. Las siguientes secciones proporcionan una pequeña introducción a Django extraida de este enlace,

4.1. Componentes de una Aplicación Web de Django

Una aplicación web Django espera peticiones HTTP del explorador web (o de otro cliente). Cuando se recibe una petición, la aplicación elabora la respuesta basándose en la URL y en la información incluida en los datos POST o GET recibidos. De-

pendiendo de qué se necesita quizás pueda entonces leer o escribir información desde una base de datos o realizar otras tareas requeridas para satisfacer la petición. La aplicación devolverá a continuación una respuesta al explorador web, con frecuencia creando dinámicamente una página HTML para que el explorador la presente insertando los datos recuperados dentro de una plantilla HTML.

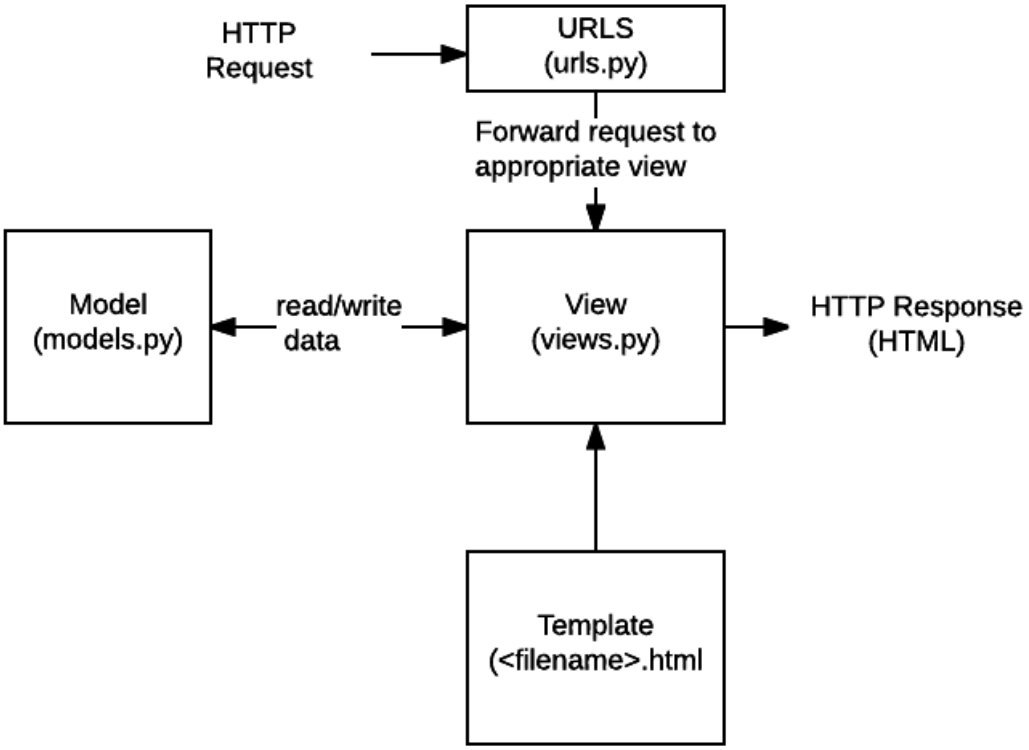

Figura 1: Esquema típico de funcionamiento de una aplicación web creada usando Django.

Las aplicaciones web de Django normalmente agrupan el código que gestiona cada uno de estos pasos en ficheros separados:

- **URLs**: Se usa un mapeador URL para redirigir las peticiones HTTP a la vista apropiada basándose en la URL de la petición. El mapeador URL se usa para redirigir las peticiones HTTP a la vista apropiada basándose en la

URL de la petición. El mapeador URL puede también emparejar patrones de cadenas o dígitos específicos que aparecen en una URL y los pasa a la función de visualización como datos.

- **Vista (view)**: Una vista es una función de gestión de peticiones que recibe peticiones HTTP y devuelve respuestas HTTP. Las vistas acceden a los datos que necesitan para satisfacer las peticiones por medio de modelos, y delegan el formateo de la respuesta a las plantillas ("templates").
- **Modelos (Models)**: Los Modelos son objetos de Python que definen la estructura de los datos de una aplicación y proporcionan mecanismos para gestionar (añadir, modificar y borrar) y consultar registros en la base de datos.
- **Plantillas (templates)**: una plantilla es un fichero de texto que define la estructura o diagrama de otro fichero (tal como una página HTML), con marcadores de posición que se utilizan para representar el contenido. Una vista puede crear dinámicamente una página web usando una plantilla, rellenándola con datos de un modelo.

La Figura 1 muestra de forma esquemática las interacciones entre estas distintas componentes.

4.2. Mapeador URL (urls.py)

Un mapeador URL está normalmente almacenado en un fichero llamado urls.py. En el ejemplo más abajo el mapeador (urlpatterns) define una lista de mapeos entre patrones URL específicos y sus correspondientes funciones de visualización. Si se recibe una Petición HTTP que tiene una URL que empareja un patrón específico se realizará una llamada y se pasará la petición a la función de visualización asociada (ej. views.index).

Listado 1: Listado de un fichero **urls.py** de ejemplo en Django

```
1 urlpatterns = [
2     url(r'^$', views.index),
3     url(r'^([0-9]+)/$', views.best),
```

```
]
```

4.3. Manejar la Petición (views.py)

Las vistas son el corazón de la aplicación web, recibiendo peticiones HTTP de los clientes web y devolviendo respuestas HTTP. Entre éstas, organizan los otros recursos del framework para acceder a las bases de datos, consolidar plantillas, etc.

El ejemplo más abajo muestra una mínima función de visualización index(), que podría ser llamada por nuestro mapeador de URL de la sección anterior. Al igual que todas las funciones de visualización, recibe un objeto HttpRequest como parámetro (request) y devuelve un objeto HttpResponse. En este caso no hacemos nada con la petición y nuestra respuesta simplemente devuelve una cadena insertada de forma fija en el código.

Listado 2: Listado de un fichero **views.py** de ejemplo en Django

```
## fichero: views.py (funciones de visualizacion de Django)
from django.http import HttpResponse

def index(request):
    # Obtener un HttpRequest - el parametro peticion
    # Realizar operaciones usando la infomracion de la peticion.
    # Devolver una HttpResponse
    return HttpResponse('!Hola desde Django!')
```

4.4. Definir Modelos de Datos (models.py)

Las aplicaciones web Django manejan y consultan datos a través de objetos Python referidos como modelos. Los modelos definen la estructura de los datos almacenados, incluyendo los tipos de campos y su tamaño máximo, los valores por defecto, la lista de selección de opciones, etc. La definición del modelo es independiente de la base de datos subyacente. Puedes elegir una entre varias como parte

de la configuración de tu proyecto. Una vez que has seleccionado qué base de datos quieres usar, no necesitas en absoluto comunicarte con ella directamente, sólo hay que escribir la estructura de la base y otro código y Django se encarga por ti de todo el trabajo sucio de comunicarse con la base de datos.

El fragmento de código de más abajo muestra un modelo de Django muy simple para un objeto Team. La clase Team deriva de la clase de django models.Model. Define el nombre de un equipo y su nivel como campos de tipo carácter y especifica un número máximo de caracteres que pueden ser almacenados en cada registro. El team_level puede ser un valor de entre varios, de manera que lo definimos como un campo de opciones y proporcionamos un mapeo entre opciones para mostrar y datos para almacenar, junto con un valor por defecto.

Listado 3: Listado de un fichero **models.py** de ejemplo en Django

```
# filename: models.py

from django.db import models

class Team(models.Model):
    team_name = models.CharField(max_length=40)

    TEAM_LEVELS = (
        ('U09', 'Under 09s'),
        ('U10', 'Under 10s'),
        ('U11', 'Under 11s'),
        ...  #list other team levels
    )
    team_level = models.CharField(max_length=3,\
            choices=TEAM_LEVELS,default='U11')
```

4.5. Consultar Datos (views.py)

El modelo de Django proporciona una API de consulta simple para buscar en la base de datos. Esta puede buscar coincidencias contra varios campos al mismo

tiempo usando diferentes criterios (ej. exacto, insensible a las mayúsculas, mayor que, etc.), y puede soportar sentencias complejas (por ejemplo, puedes especificar que se busque equipos U11 que tengan un nombre de equipo que empiece por 'Fr' o termine con 'al').

El fragmento de código de más abajo muestra una función de visualización (manejador de recursos) para presentar en pantalla todos nuestros equipos U09. La línea 7 muestra como podemos usar la API de consulta del modelo para filtrar todos los registros donde el campo team_level tenga exactamente el texto 'U09' (fíjate como este criterio se pasa como argumento a la función filter() con el nombre del campo y tipo de coincidencia separados por un doble guion bajo: team_level__exact).

Listado 4: Listado de un fichero **views.py** de ejemplo en Django

```
1 ## filename: views.py
2
3 from django.shortcuts import render
4 from .models import Team
5
6 def index(request):
7     list_teams =
        ↪ Team.objects.filter(team_level__exact="U09")
8     context = {'youngest_teams': list_teams}
9     return render(request, '/best/index.html', context)
```

4.6. Renderización de los Datos (plantillas HTML)

Los sistemas de plantillas permiten especificar la estructura de un documento de salida usando marcadores de posición para los datos que serán rellenados cuando se genere la página. Las plantillas se usan con frecuencia para crear HTML, también pueden crear otros tipos de documentos.

El fragmento de código de más abajo muestra el aspecto que podría tener la plantilla HTML llamada por la función render() de la sección anterior. Esta plantilla ha sido escrita bajo la suposición de que cuando sea renderizada tendrá acceso a una variable tipo lista llamada youngest_teams (contenida en la variable context

dentro de la función render() de más arriba). Dentro del esqueleto HTML tenemos una expresión que primero comprueba que existe la variable youngest_teams, y luego itera sobre ella en un bucle for. En cada iteración la plantilla presenta cada valor del campo team_name del equipo en un elemento <li>.

Listado 5: Listado de un fichero **index.html** de ejemplo en Django

```
## filename: best/templates/best/index.html

<!DOCTYPE html>
<html lang="en">
<body>

 {% if youngest_teams %}
    <ul>
    {% for team in youngest_teams %}
        <li>{{ team.team_name }}</li>
    {% endfor %}
    </ul>
{% else %}
    <p>No teams are available.</p>
{% endif %}

</body>
</html>
```

5. Gunicorn

Gunicorn (Green Unicorn) es un servidor HTTP basado en Python que se utiliza ampliamente para implementar aplicaciones web desarrolladas con frameworks como Django o Flask. Diseñado para ser eficiente, ligero y compatible con la especificación WSGI (Web Server Gateway Interface), Gunicorn permite manejar múltiples solicitudes simultáneamente utilizando un modelo de procesos múltiples o de múltiples hilos. Su configuración sencilla y su capacidad para integrarse con servidores

proxy como Nginx lo convierten en una solución ideal para entornos de producción, ofreciendo alto rendimiento y estabilidad en el despliegue de aplicaciones Python.

Una vez instalado Gunicorn, estará disponible el comando gunicorn, que inicia el proceso del servidor Gunicorn. La forma más sencilla de utilizarlo es pasando la ubicación de un módulo que contiene un objeto de aplicación WSGI. A la hora de arrancar gunicorn como servidor web, es necesario únicamente ejecutarlo como un proceso, pasando ciertos argumentos de configuración. Por ejemplo,

```
gunicorn --bind 0.0.0.0:8000 nombreProyecto.wsgi:aplicacion
```

Esto iniciará un proceso con un hilo, escuchando en 0.0.0.0:8000 sirviendo solicitudes que atenderá la aplicación especificada como 'aplicación'. Es decir Gunicorn escuchará por todas las interfaces, en el puerto 8000. Es necesario que tu proyecto Django esté en el Python path. La forma más sencilla de asegurarlo es ejecutar este comando desde el mismo directorio donde se encuentra tu archivo manage.py, en tu proyecto Django. Además de esto, gunicorn soporta varias opciones que están relacionadas con el nº de procesos que se utilizarán para servir solicitudes, cuántos hilos usará cada proceso, etc. Para más información ejectuar:

```
gunicorn --help
```

y leer la documentación.

También es posible configurar Gunicorn como un servicio del sistema. Esto se explicará en las prácticas siguientes.

6. PostgreSQL

En el entorno de las prácticas se empleará PostgreSQL como gestor de bases de datos.

A lo largo del curso se asume que el alumno ya conoce y maneja dicho gestor de bases de datos, así como las siguientes herramientas de línea de comandos:

- createdb: Comando para crear una nueva base de datos. Al igual que los siguientes comandos, empleará el parámetro -U para indicar qué usuario se quiere emplear. Para crear la base de datos "DB" se deberá ejecutar:

```
createdb -U alumnodb DB
```

- psql: Cliente de línea de comandos para acceder a la base de datos. Por ejemplo, para conectarnos a las base de datos "DB" con el usuario de PostgreSQL alumnodb previamente creado, escribiremos:

  ```
  psql -U alumnodb DB
  ```

- dropdb: Comando para eliminar una nueva base de datos. Sigue el patrón de los comandos anteriores. Para borrar la base de datos "DB" se deberá ejecutar:

  ```
  dropdb -U alumnodb DB
  ```

  En caso de que el servidor PostgreSQL no se encuentre en el host local, y no se ejecute en el puerto por defecto (5432) será necesario especificar las opciones:

  ```
  -h 10.X.Y.Z -port N
  ```

  Se recomienda probar a ejecutar estos comandos contra el servidor PostgreSQL de la MV, utilizando la dirección IP y número de puerto correspondiente. Para ello, se recomienda volver a consultar la tabla de mapeo de puertos de la MV.

6.1. DBeaver

Dbeaver es una potente herramienta de administración y desarrollo con bases de datos, para conectar a múltiples DBMS. A lo largo de estas prácticas podremos optar por cualquier herramienta gráfica, aunque a continuación presentamos dbeaver como una alternativa para ello.

Antes de probar la conexión con DBeaver siguiendo los pasos descritos a continuación, es necesario que creemos una base de datos de nombre DB para el usuario alumnodb en la máquina virtual. Para ello, deberemos utilizar el comando createdb descrito anteriormente. El usuario alumnodb ya está creado en PostgreSQL en la máquina virtual.

Para conectar al gestor de bases de datos iniciaremos la aplicación dbeaver desde el menú de aplicaciones, o bien escribiendo dbeaver desde la línea de comandos. A continuación crearemos una nueva conexión a una DB usando el botón que tiene forma de enchufe o mediante el menú Database→New Database Connection.

Seleccionamos PostgreSQL como gestor de la DB y se nos mostrará un diálogo para acceder a la base de datos, similar al mostrado en la figura 2.

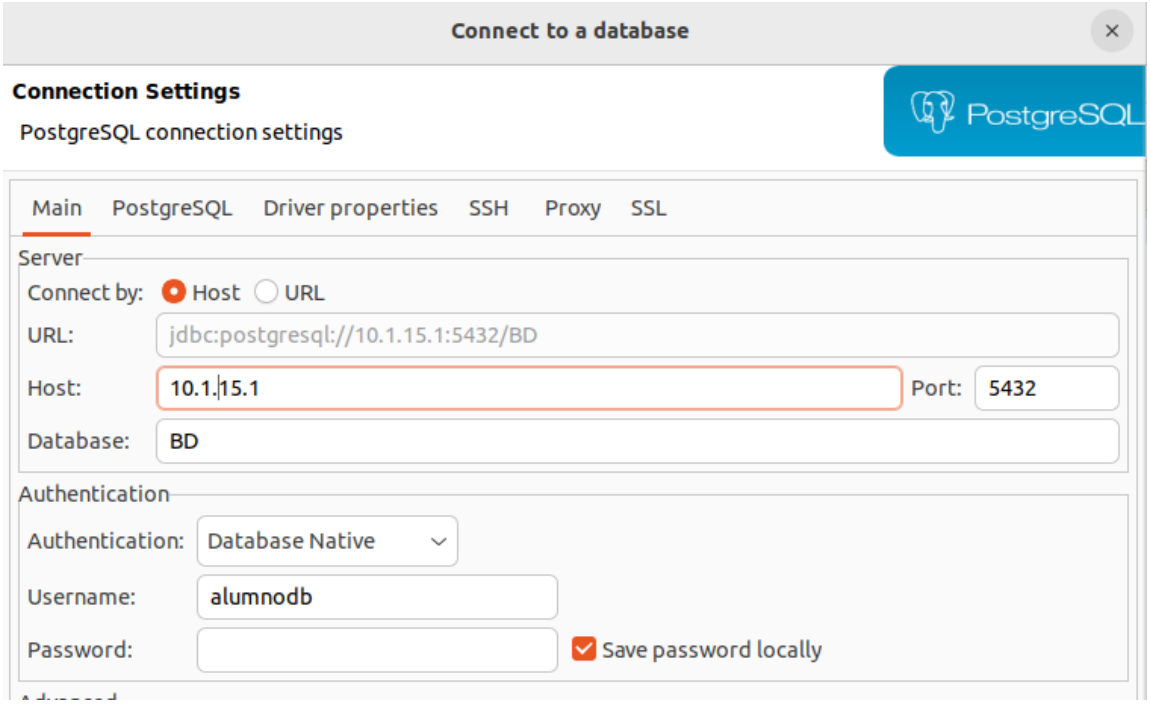

Figura 2: Conectando a PostgreSQL con DBeaver.

Deberemos:

- Indicar el nombre de la base de datos: BD.
- Indicar el nombre de usuario: alumnodb.
- Indicar la contraseña, en nuestro caso, vacía.
- Indicar el host de conexión: será la IP de la máquina virtual en la que se esté ejecutando postgresql.

- Indicar el número de puerto donde esté escuchando PostgreSQL (dependerá de la MV).
- Pulsar OK.

Si todo va bien, podemos navegar por la conexión recién creada Databases->BD->Schemas->public->Tables... También podemos abrir el editor de SQL en una consola mediante el desplegable con texto SQL o el menú SQL Editor→ Open SQL Console. Ver figura 3

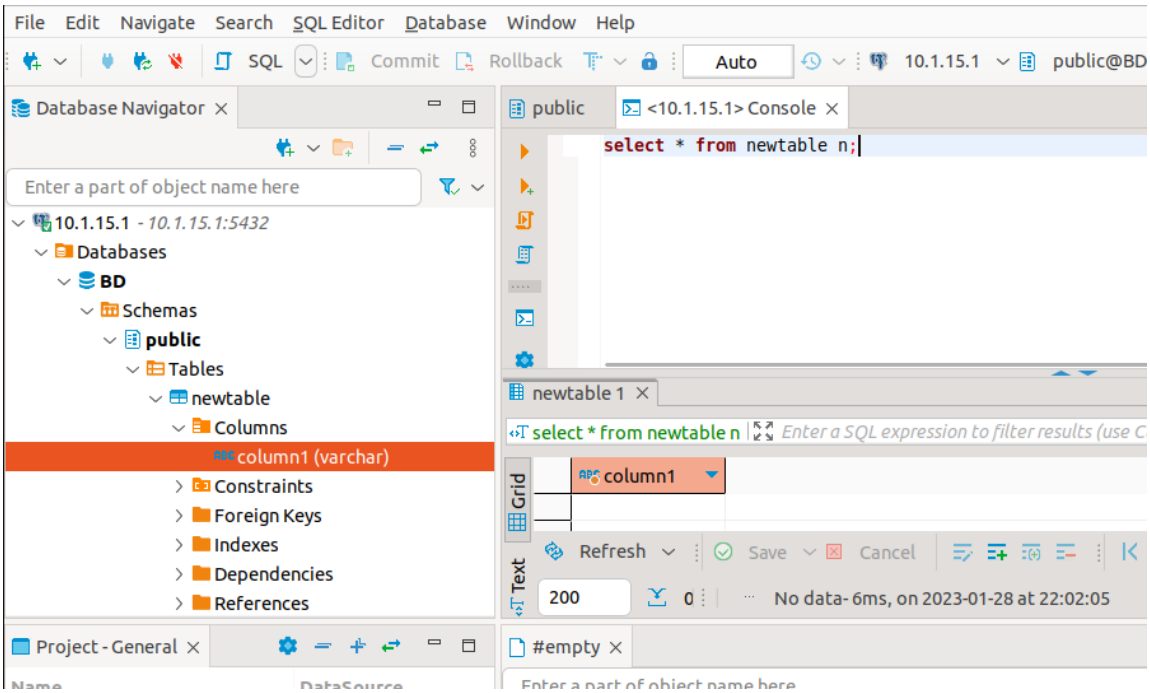

Figura 3: Vista de editor SQL en DBeaver.

Haremos uso de las siguientes dos vistas de la base de datos:

- Editor SQL (SQL Editor): Permite ejecutar consultas SQL. Esto se realiza seleccionando la consulta y pulsando en los iconos triangulares de la esquina superior izquierda, para ejecutar una consulta (bajo el cursor) o todas las consultas a partir del cursor. También se puede hacer lo mismo seleccionando texto y pulsando Ctrl-Enter.

- Navegador del la DB (Database Navigator). Permite revisar la estructura de tablas y columnas, realizando modificaciones en dicho esquema. Para ver las tablas de nuestra BD hay que seleccionar el schema public.

Se recomienda probar a conectarse usando DBeaver contra el servidor PostgreSQL de la MV, utilizando la dirección IP y número de puerto correspondiente. Para ello, se recomienda volver a consultar la tabla de mapeo de puertos de la MV.

7. Apache Jmeter

Apache JMeter es una herramienta de código abierto desarrollada por la Apache Software Foundation, diseñada para realizar pruebas de carga y medir el rendimiento de diversos servicios, con un enfoque particular en aplicaciones web.

Es una aplicación 100 % Java que permite a los usuarios simular múltiples usuarios concurrentes, generando cargas pesadas en servidores, redes o aplicaciones para evaluar su fortaleza y analizar su rendimiento bajo diferentes tipos de carga.

JMeter es capaz de probar el rendimiento tanto de recursos estáticos como dinámicos, permitiendo la simulación de una carga pesada en un servidor, grupo de servidores, red u objeto para probar su fortaleza o analizar el rendimiento general bajo diferentes tipos de carga.

Además, JMeter puede ser utilizado para pruebas funcionales de aplicaciones, verificando que las respuestas del servidor sean correctas y que las aplicaciones funcionen como se espera bajo diferentes condiciones.

En resumen, Apache JMeter es una herramienta versátil y poderosa para realizar pruebas de carga y rendimiento en una amplia variedad de servicios y aplicaciones, siendo especialmente útil en el ámbito de las aplicaciones web.

La figura 4 ilustra la arquitectura de Apache Jmeter.

8. Servidor Apache

El servidor HTTP Apache es un servidor web de código abierto desarrollado por la Apache Software Foundation. Compatible con múltiples plataformas, incluyendo

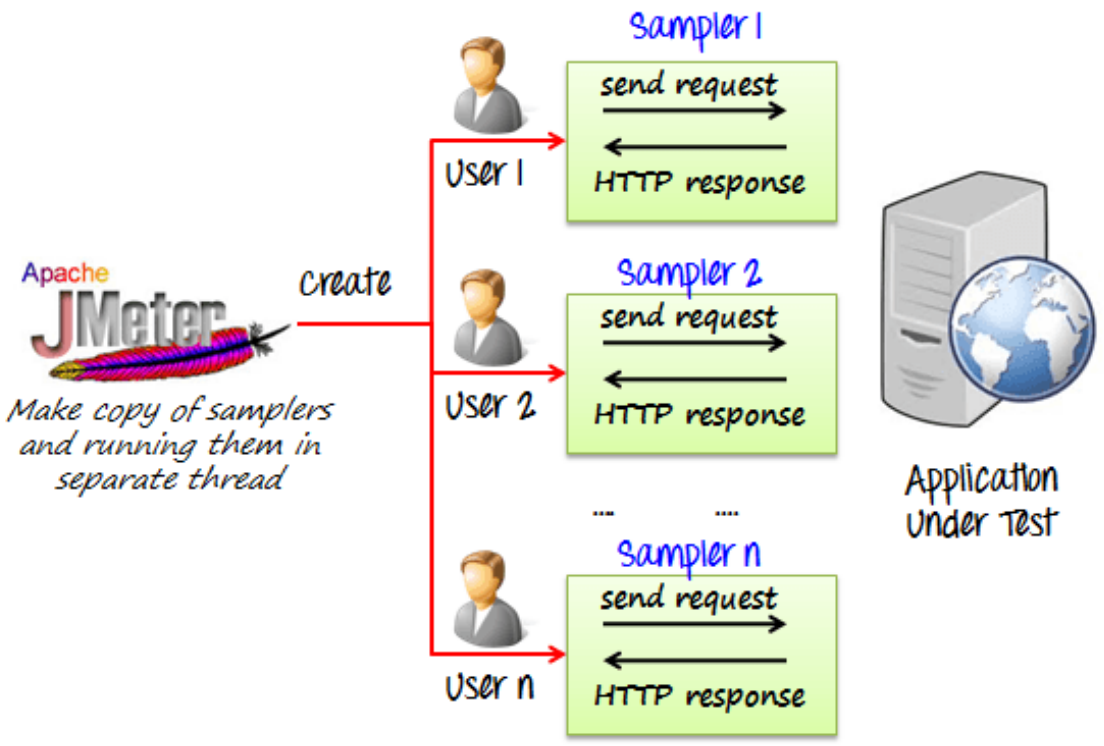

Figura 4: Esquema de funcionamiento de Apache Jmeter extraido de este enlace.

Unix (BSD, GNU/Linux), Windows y macOS, implementa el protocolo HTTP/1.1 y soporta la funcionalidad de sitios virtuales según la normativa RFC 2616. Su desarrollo comenzó en 1995, inicialmente basado en el código del NCSA HTTPd 1.3, pero posteriormente fue reescrito por completo. El nombre *Apache* se eligió en honor a la tribu nativa americana Apache, simbolizando firmeza y resistencia.

Apache es altamente configurable y modular, permitiendo la integración de diversas funcionalidades a través de módulos adicionales. Aunque en sus inicios fue criticado por la falta de una interfaz gráfica para su configuración, su flexibilidad y robustez le han permitido mantener una amplia aceptación en la web. Desde 1996, ha sido uno de los servidores HTTP más utilizados, desempeñando un papel fundamental en el desarrollo de la World Wide Web. Alcanzó su máxima cuota de mercado en 2005, siendo empleado en el 70 % de los sitios web a nivel mundial.

En resumen, el servidor HTTP Apache ha sido una pieza clave en la evolución

de internet, ofreciendo una solución robusta y flexible para el alojamiento de sitios web y aplicaciones en diversas plataformas.

8.1. Apache Como Balanceador de Carga

El servidor Apache HTTP (httpd) no solo es capaz de servir contenido estático y dinámico directamente a los usuarios finales, sino que también puede funcionar como un servidor proxy inverso, también conocido como un balanceador de carga. En este rol, Apache no aloja ni genera el contenido directamente. En su lugar, obtiene el contenido de uno o más servidores backend, que normalmente no son accesibles directamente desde redes externas. Cómo funciona:

- **Solicitud del cliente**: Un cliente envía una solicitud a Apache.
- **Proxy al backend**: Apache reenvía la solicitud a uno de los servidores backend.
- **Generación de contenido**: El servidor backend procesa la solicitud, genera el contenido requerido y lo envía de vuelta a Apache.
- **Entrega de la respuesta**: Apache da formato a la respuesta y la entrega al cliente, haciendo que parezca que el contenido se originó directamente del servidor Apache.
- **Alta disponibilidad**: Asegura un servicio continuo al enrutar las solicitudes a múltiples servidores backend.
- **Balanceo de carga**: Distribuye el tráfico entre los servidores backend para evitar sobrecargas.

En estas configuraciones, el balanceador actúa como el único punto de contacto para los clientes, proporcionando una capa adicional de abstracción y protección para la infraestructura backend. La figura 5 describe el esquema de funcionamiento de Apache como balanceador de carga. En las prácticas de SI2 nos centraremos en el uso del servidor Apache como balanceador de carga.

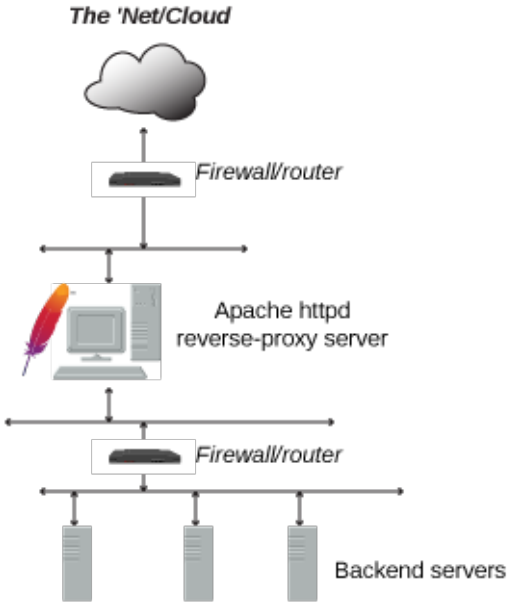

Figura 5: Esquema de funcionamiento de Apache como balanceador de carga extraido de este enlace.

Para probar que Apache funciona correctamente podemos acceder al puerto correspondiente de la MV desde el navegador. Revisar de nuevo la tabla de mapeo de puertos de la MV.

9. Instalación del Entorno en un PC Propio

En esta sección se describe cómo instalar todos los componentes que son necesarios para llevar a cabo las prácticas de SI2. Se supone que el estudiante tiene ya instalada una distribución de linux como Ubuntu 22.04 o Ubuntu 24.0.

9.1. Virtual Box

Para instalar virutal box basta con acceder a `https://www.virtualbox.org/` y descargar el paquete debian correspondiente a la distribución elegida. Por ejemplo

Ubunut 22.04. Se instala mediante el comando:

```
sudo dpkg -i virtualbox-7.1_7.1.4-165100~Ubuntu~jammy_amd64.deb
```

Las herramientas para generar claves públicas y privadas, se pueden instalar con:

```
sudo apt-get update
sudo apt-get install openssh-client
```

También será necesario instalar el paquete de gestión de imágenes de máquinas virtuales.

```
sudo apt-get install cloud-image-utils
```

9.2. Python 3

Django corre sobre python versión 3, por lo que es necesario instalar estas componentes antes. Para ello, ejecutaremos los comandos:

```
sudo apt-get update
sudo apt-get install python3-venv python3-pip
```

Estos comandos instalaran una instalación básica de python y el sistema para crear entornos virtuales de python. Una vez hayamos instalado estos paquetes, podemos crear un entorno virtual de python con:

```
python3 -m venv ruta_entorno_virtual
```

donde ruta_entorno_virtual es la carpeta donde queremos crear el entorno virtual de python. Para acceder sin problema a dicho entorno y usar la versión de python y pip correspondiente, basta con actualizar la variable de entorno PATH:

```
export PATH=ruta_entorno_virtual/bin:$PATH
```

Este comando se puede introducir en el fichero .bashrc del directorio HOME del usuario para no tener que introducirlo cada vez que se abre una terminal.

9.3. Django, Gunicorn y Demás Paquetes Python

Los requisitos de paquetes python para llevar a cabo las prácticas se pueden guardar en un fichero llamado requirements.txt con:

```
cat > requirements.txt << "EOF"
asgiref==3.8.1
beautifulsoup4==4.12.3
certifi==2024.8.30
charset-normalizer==3.4.0
dj-database-url==2.2.0
Django==5.2.2
django-modern-rpc==1.0.3
djangorestframework==3.15.2
Faker==30.8.0
flake8==6.1.0
google==3.0.0
grpc-django==1.0.0
grpcio==1.67.1
grpcio-tools==1.67.1
gunicorn==20.1.0
idna==3.10
protobuf==5.28.3
psycopg2-binary==2.9.9
python-dateutil==2.9.0.post0
python-dotenv==0.21.0
requests==2.32.3
six==1.16.0
soupsieve==2.6
sqlparse==0.5.1
typing_extensions==4.12.2
urllib3==2.2.3
whitenoise==6.7.0
```

```
pika==1.3.2
EOF
```

Estos paquetes se pueden instalar con pip con:

```
pip install -r requirements.txt
```

9.4. Jakarta Jmeter

Este programa es una herramienta de generación de carga y testing que usaremos en posteriores prácticas. De momento, si queremos instalarlo en nuestro PC particular, descargaremos el paquete jakarta-jmeter-5.6.3.tgz de `http://jmeter.apache.org/download_jmeter.cgi`. Eligiendo de entre las opciones la versión binaria .5.6.3.tgz: `http://apache.cbox.biz/jmeter/binaries/apache-jmeter-5.6.3.tgz`. A continuación lo descomprimimos en la ruta de nuestra elección, por ejemplo, la carpeta personal, mediante el comando:

```
cd $HOME
tar -xzvf apache-jmeter-5.6.3.tgz
```

Jmeter funciona con java, que debe ser instalado para poder utilizarlo:

```
sudo apt-get update
sudo apt-get install default-jre
```

Para ejecutarlo lanzamos el script jmeter.sh:

```
~/apache-jmeter/bin/jmeter.sh
```

9.5. DBeaver

Para instalar DBeaver se puede descartar desde este enlace el paquete correspondiente formato Debian, que se puede instalar con el comando:

```
sudo dpkg -i dbeaver-ce_24.3.2_amd64.deb
```

9.6. Otras Herramientas

Además del software mencionado anteriormente, resultará conveniente instalar otras herramientas que ayudarán a llevar a cabo las prácticas. Estas se pueden instalar con el siguiente comando:

```
sudo apt-get update
sudo apt-get install postgresql-client net-tools nmon
```

10. Entrega

Como ya se ha indicado, no es necesario entregar nada en esta práctica. Sin embargo, se espera que el alumno instale las herramientas necesarias en su ordenador personal.